

Summary of Changes

Dear AC and Reviewers:

We thank the AC and reviewers for their constructive feedback and for the opportunity to revise and resubmit our paper. The revision focuses on making composition and TinyEdge mining concrete, isolating the sources of G-Agent’s efficiency gains, evaluating the method in a large-scale realistic API ecosystem, and characterizing empty-memory online operation, backbone robustness, and planning-memory scalability.

Major Revisions

- **Made TinyEdge mining and composition concrete.**
We rewrote the method section and added an algorithm covering complete-workflow retrieval, coarse planning, Persistent TinyEdge selection, executable subplan expansion, ordered assembly, and base-LLM fallback. We also formalize canonical operations, workflow hyperedges, fragment mining criteria, support and provenance, incremental mining, and online admission, improving reproducibility.
- **Separated structural reuse from runtime batching.**
We distinguish Persistent TinyEdges from Transient TinyEdge Groups. Persistent TinyEdges factorize fragments shared across historical workflows and support partial composition and compact storage. Transient TinyEdge Groups are runtime-only structures for shared-prompt processing. We add single-switch ablations for workflow-hyperedge reuse and composition, together with a method-level no-memory control, accepted-reuse quality analysis, and representative Composition traces.
- **Added a large-scale realistic tool-use benchmark.**
To evaluate G-Agent beyond relatively small tool ecosystems, we add AppWorld, a large-scale interactive benchmark with a broader and more heterogeneous application and API ecosystem. We now evaluate G-Agent across four benchmarks, add a three-backbone TravelPlanner robustness analysis, and discuss where the gains generalize or weaken.
- **Added sensitivity, empty-memory online evaluation, and scalability analyses.**
We add an empty-memory online evaluation protocol, Fast Path threshold sensitivity, request-side shared-prompt analysis, and memory-side scaling to 500K plan-step references. We also specify workflow-level LRU maintenance to bound persistent memory and consistently update TinyEdge support and provenance after eviction.

Response to Major Concerns

Concern 1: Composition and Persistent TinyEdge mining were insufficiently specified.

Response. Section 4.4 now formalizes executable-fragment extraction and TinyEdge materialization. Sections 4.3–4.5 and Algorithm 1 describe workflow reuse, TinyEdge retrieval and selection, composition conditions, lightweight expansion, ordered assembly, and fallback. We also define support, closedness, executability, provenance, and the complexity of incremental mining.

Concern 2: The gains might arise from generic reuse or batching rather than the proposed representation.

Response. We narrow our claim and do not argue that fragment reuse is exclusive to hypergraphs. Instead, the hypergraph provides a unified representation of complete workflows, shared fragments, execution order, support, and provenance. Shared-prompt batching remains a separate runtime optimization. Our ablation shows that Composition increases the hit rate from 42.22% to 83.33% and reduces cost by 18.4%, but does not improve accuracy. *Location: Sections 4.3–4.6 and 5.4; Figure 5; Table 4.*

Concern 3: Evaluation in a large-scale, realistic tool ecosystem was missing.

Response. We add AppWorld, a large-scale benchmark with a broad, heterogeneous application and API ecosystem. The four-benchmark evaluation shows cost reductions, and a three-backbone TravelPlanner study tests robustness. We also report weaker gains on tasks dominated by query- or instance-specific reasoning. *Location: Sections 5.1–5.2 and 5.5; Tables 2 and 5.*

Concern 4: TinyEdge benefits, hit-rate interpretation, and failure behavior were insufficiently validated.

Response. We report accepted-reuse rates and conditional task effectiveness for reused and non-reused requests. We compare reuse quality with PlanCache across all four benchmarks and evaluate Composition and shared-prompt processing separately. Appendix B includes successful and failed Composition cases, while Figure 8 summarizes the Fast, Composition, and LLM-fallback routes together with online memory updates. *Location: Sections 4.5–4.6 and 5.2–5.4; Tables 3 and 4; Figures 6 and 8; Appendix B.*

Concern 5: Cold start, thresholds, memory growth, retrieval overhead, and stale plans required further analysis.

Response. G-Agent starts from **empty memory** and updates it online, accounting for cold-start overhead. We distinguish fragment-mining criteria from workflow and TinyEdge retrieval thresholds. At 500K plan-step references, TinyEdge-Compact reduces storage by 50.8% and adds only 0.100 ms to p95 retrieval-and-reconstruction latency. LRU bounds memory growth, while abrupt interface changes remain a limitation. *Location: Sections 5.1 and 5.3; Figures 6–7; Appendices C–D.*

Concern 6: Candidate selection and memory maintenance were unclear.

Response. We now specify that Fast reuse selects the highest-scoring workflow above its threshold, while Composition selects the highest-scoring threshold-qualified TinyEdge for each coarse subgoal. Admitted trajectories update planning memory, and workflow-level LRU removes the least recently used workflows while updating TinyEdge support and provenance. We also clarify that abrupt interface changes may require explicit invalidation beyond LRU. *Location: Sections 4.5 and 4.7; Appendix D and Algorithm 2.*

Concern 7: Presentation inconsistencies and related-work omissions reduced clarity.

Response. We corrected pseudocode and cross-reference inconsistencies, standardized the workflow, TinyEdge, route, and metric terminology, revised the framework figures, added a complete workflow overview, and added SagaLLM to the related-work discussion. *Location: Sections 2.2–2.3 and 4; Figures 1, 4, 5, and 8.*

G-Agent: Efficient LLM Agent with Hypergraph Planning

Anonymous Author(s)

Abstract

LLM agents repeatedly invoke large models to plan multi-step tool-use workflows, making planning a major bottleneck in cost and latency. We observe substantial redundancy: while full plans vary, recurring plan fragments appear across requests and across alternative valid solutions. We propose hypergraph-based planning memory, representing steps as nodes and plans as hyperedges, and introduce tiny edges to factorize frequent shared substructures for efficient retrieval and composition. Based on this memory, G-Agent plans by retrieving and assembling structures from the hypergraph, falling back to LLM planning only when needed and then storing new plans for future reuse. Experiments across diverse tool-use workloads show that G-Agent substantially reduces planning calls and end-to-end cost while maintaining competitive success rates.

CCS Concepts

• Computing methodologies → Artificial intelligence.

Keywords

Large Language Model, Agent, Hypergraph

ACM Reference Format:

Anonymous Author(s). 2018. G-Agent: Efficient LLM Agent with Hypergraph Planning. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Large Language Model (LLM) agents have emerged as a powerful interface for solving complex, open-ended tasks that require multi-step reasoning, tool usage, and interaction with external environments [6, 10, 25, 31]. From a data mining perspective, agent planning is not purely a generative reasoning problem; it is also a *structure mining and reuse* problem. Execution traces contain recurring plan patterns that can be indexed, retrieved, and recomposed, analogous to frequent pattern and sequence mining in large data collections [1, 9, 15, 18, 29]. This view is consistent with recent data-management trends on high-throughput vector similarity search and vector-aware database systems, where indexing and query efficiency are first-class concerns [11, 16, 33]. Modern agents typically operate in a plan-act-observe loop: a user request is decomposed into sub-tasks, executed via tools, and iteratively revised based on feedback [31].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

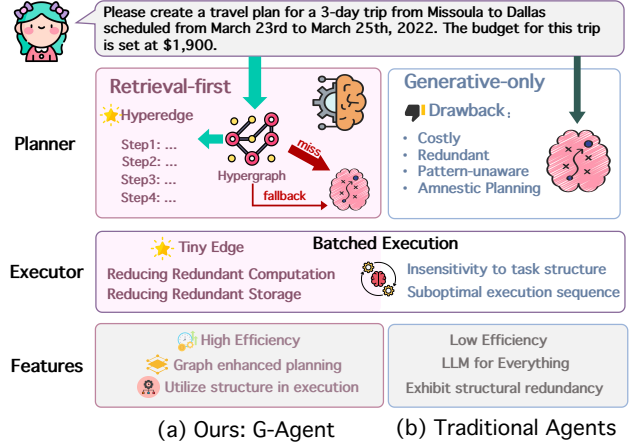


Figure 1: Comparison between G-Agent and traditional agents.

However, current agent architectures pay a steep price for this generality. In most systems, planning typically handles the most challenging decisions in task execution and therefore often relies on capable, expensive models. Yet most systems treat each planning problem as a fresh generation, repeatedly invoking the LLM across requests and accumulating substantial call and token costs [21, 30, 31]. This “LLM-for-everything” pipeline is misaligned with cost-efficient problem solving: humans do not deeply deliberate for every routine step; they retrieve familiar procedures for recurring sub-problems and reserve slow, expensive reasoning for true novelty. This motivates a different design principle for efficient agents: systematically reuse structured planning knowledge and invoke costly LLM planning only when necessary.

Our work is built on a key empirical observation: agent workloads exhibit substantial **structural redundancy at multiple granularities**. If an agent has already solved semantically similar sub-problems, repeatedly prompting a high-capacity LLM to regenerate nearly identical planning structure is wasteful. This reframes planning from a purely generative process into a retrieval-and-composition process: Can historical planning structure be externalized for fast reuse, while preserving robustness on novel cases?

To answer this, we propose a new agent architecture named G-Agent that inserts a structured planning memory between user requests and LLM planning as shown in Figure 1. The core representation is a hypergraph: nodes represent atomic sub-task steps, and each historical plan is stored as a hyperedge connecting the set of steps that jointly realize a goal. Compared with sequential plan libraries or template trees, hypergraphs naturally capture higher-order compositionality, where step groups are shared across many plans and can be partially reused when no full-plan match exists [3, 4, 8, 34].

Crucially, we introduce **Persistent TinyEdges**: frequent shared substructures mined from overlaps among workflow hyperedges. A Persistent TinyEdge captures a cohesive reusable fragment (e.g., a *search-filter-verify* routine) that appears across many plans. The two TinyEdge forms provide complementary benefits. Persistent TinyEdges factorize common structure, reducing redundant storage and supporting efficient retrieval [11, 16] and partial-plan composition. At runtime, compatible requests associated with the same local structure may form **Transient TinyEdge Groups**, which share common prompt prefixes and amortize prompt processing across requests.

Built on this representation, we present **G-Agent**, a retrieval-first agent. Given a request, G-Agent first attempts to reuse a complete workflow. If no workflow is reused, a lightweight model generates a coarse plan, retrieves relevant TinyEdges for its subgoals, and expands the selected fragments into a detailed plan. When memory coverage is insufficient, G-Agent invokes the base LLM planner to generate a complete plan. The resulting trajectory may then be admitted to the hypergraph, enabling continual memory growth and improving future reuse.

This architecture follows an efficient cognitive pattern: *fast retrieval for familiar cases, adaptive reasoning for novel cases*. More broadly, G-Agent treats planning traces as persistent, mineable structures rather than disposable intermediate text, allowing planning effort to be amortized across requests.

Our contributions are as follows:

- We propose a retrieval-first planning architecture that inserts **hypergraph memory** between requests and the LLM, reducing redundant generative planning calls.
- We model planning knowledge as hypergraph structures and introduce **Persistent TinyEdges** to factorize frequent fragments shared across workflows, enabling partial-plan composition and compact storage.
- We design G-Agent to combine complete workflow reuse, coarse-plan-guided TinyEdge composition, base LLM planning, and online memory updates, with **Transient TinyEdge Groups** for shared-prompt batching.
- Extensive experiments across four agent workloads show that G-Agent substantially reduces end-to-end cost while maintaining competitive task effectiveness.

2 Background and Related Work

2.1 Hypergraph and Pattern Mining

A *hypergraph* generalizes a standard graph by allowing an edge to connect an arbitrary number of vertices. Formally, a hypergraph is defined as $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a set of vertices and $\mathcal{E} \subseteq 2^{\mathcal{V}}$ is a set of *hyperedges*, each hyperedge $e \in \mathcal{E}$ being a subset of vertices [4].

While standard graphs model pairwise relationships, hypergraphs are well suited to modeling *higher-order correlations* where a group of entities operates as a functional unit [3, 8, 34]. In the context of agent planning, this property is critical: a sub-task may involve a specific combination of tool calls, constraints, and dependencies that should be represented jointly. Unlike simple graphs that fragment this relationship into pairwise links, a hyperedge

Table 1: Comparison with baselines. G-Agent excels in *batching* and *compositionality* without requiring training.

Method	Memory	Reuse Unit	Compos.	Batching	Train Free
ReAct [31]	None	N/A	Low	✗	✓
Plan-and-Solve [26]	None	N/A	Low	✗	✓
SwiftSage [14]	Weights	Policy	Low	✗	✗
PlanCache [32]	Cache	Templates	Med.	✗	✓
G-Agent	Hypergraph	Tiny Edges	High	✓	✓

naturally encapsulates the complete sub-task structure as a reusable unit.

Frequent Pattern Mining. Frequent pattern mining [1] is a foundational technique in data mining that discovers recurring structures within large datasets. Representative frequent subgraph mining algorithms, such as gSpan [29], identify sub-topologies that appear with high support in graph databases. G-Agent draws on this general perspective but does not mine arbitrary graph substructures. Instead, it incrementally extracts contiguous executable fragments from historical plans and materializes frequent, closed, and executable fragments as Persistent TinyEdges for efficient reuse.

2.2 LLM Agent Planning

LLM agents combine language models with planning, tools, and memory to interact with external environments [25]. Modular systems such as MRKL connect LLMs with specialized reasoning modules [12], while Toolformer learns when and how to invoke external tools [20]. **ReAct** interleaves reasoning and acting [31], whereas **Plan-and-Solve** generates an explicit plan before execution [26]. These methods improve tool use and decomposition, but generally regenerate planning structures for recurring requests.

Search and Feedback. Tree-of-Thoughts explores and evaluates multiple intermediate reasoning paths to improve deliberative problem solving [30], while Reflexion uses verbal feedback and episodic memory to improve agent behavior across trials [21]. ReWOO separates initial planning from subsequent evidence collection, reducing repeated prompt growth during tool use [28]. These methods improve reasoning quality or inference efficiency, but they do not externalize planning structures into a compositional memory shared across requests.

Dual-Process Architectures. Inspired by the “System 1 vs. System 2” theory of human cognition, **SwiftSage** [14] combines Swift, a small policy fine-tuned on domain-specific trajectories, with Sage, a large LLM for deliberate planning and exception handling. G-Agent adopts a similar large-small model collaboration, but builds its fast path on external planning memory rather than a task-specific trained policy. Its lightweight model generates coarse plans and expands retrieved fragments, while the base LLM planner handles requests that are not sufficiently covered by memory.

Complementary work studies reliability within ongoing agent workflows. SagaLLM [5] combines persistent context management, independent validation, and transaction-style recovery for multi-agent planning, whereas G-Agent focuses on mining and reusing planning structures across requests. SEARL [7] jointly optimizes

agent policy and tool graph memory for self-evolving agents, whereas G-Agent focuses on reusing hypergraph-structured planning fragments across requests.

2.3 Experience Reuse

Generative Agents [17] store textual experiences and retrieve them to guide later behavior, while Voyager [24] maintains an expanding library of executable skills. These systems demonstrate the value of persistent memory, but their reused units are primarily complete experiences, textual memories, or individual skills rather than shared planning fragments.

Plan Caching and Reuse. Recognizing the redundancy in agent workloads, memory-based planning approaches have emerged. Closest to our work is **PlanCache** [32], which extracts and caches structured plan templates for reuse. However, PlanCache primarily retrieves a complete template for a similar request. This template-level reuse lacks granularity when a new request requires fragments from different historical plans. G-Agent instead represents complete plans as workflow hyperedges and mines Persistent TinyEdges shared across workflows, enabling partial composition and compact storage. Transient TinyEdge Groups further provide shared-prompt batching among compatible active requests. Retrieval-of-Thought [2] retrieves and recomposes past reasoning steps via a thought graph, but at the step level rather than structured planning fragments. From Reasoning Traces to Reusable Modules [13] shows RL can decompose reasoning traces into reusable atomic modules, providing a theoretical lens complementary to G-Agent’s fragment-level composition.

Positioning of G-Agent. As summarized in Table 1, G-Agent fills a distinct gap. Unlike ReAct and Plan-and-Solve, which repeatedly rely on generative planning, and unlike SwiftSage, which requires task-specific fine-tuning, G-Agent offers a training-free, retrieval-first architecture. Compared with PlanCache, G-Agent supports fragment-level composition when no complete workflow can be reused. Its Persistent TinyEdges capture structure shared across historical workflows, while Transient TinyEdge Groups support shared-prompt processing across active requests.

3 Motivation

LLM agents are commonly built around a plan-act-observe loop, where an LLM generates and revises plans throughout task execution. This design is robust but expensive, because planning structures are repeatedly regenerated even when similar problems have been solved before. It is also misaligned with human intuition: people retrieve familiar procedures for recurring situations and reserve effortful reasoning for genuinely novel problems. Motivated by this perspective, we analyze agent plans and identify two forms of structural redundancy that support a retrieval-first planning architecture.

3.1 Insight 1: Plan-Level Redundancy

Insight 1. Distinct real-world requests frequently induce identical or near-identical planning structures, enabling complete-workflow reuse instead of planning from scratch.

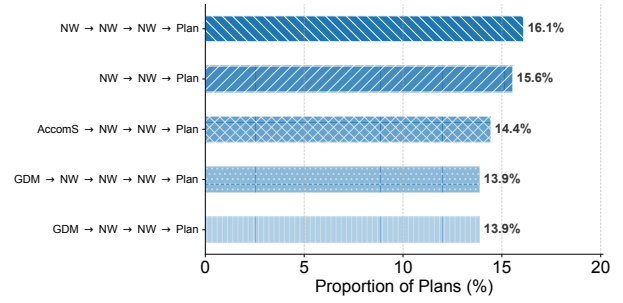


Figure 2: Top-5 near-identical plan patterns. Each bar shows the percentage of plans whose action sequence is within edit distance 1 of the given pattern.

If different requests lead to the same planning skeleton, repeatedly asking an LLM to regenerate that skeleton is unnecessary. Historical workflows can provide the reusable task structure, while the current request supplies cities, dates, preferences, and other runtime values. Complete-workflow reuse can therefore avoid expensive planning without limiting the system’s ability to handle unfamiliar requests.

We collect 180 requests from TravelPlanner [27] and obtain one trajectory for each request using a ReAct-style LLM agent. The agent uses tools including *lightSearch*, *GoogleDistanceMatrix*, *NotebookWrite*, and *Planner*, and each plan is represented by its action sequence. For each unique sequence P , we measure the fraction of plans whose edit distance from P is at most one, allowing one insertion, deletion, or substitution, and report the five patterns with the highest near-identical coverage.

As shown in Figure 2, the top patterns each cover 13.9–16.1% of all plans (NW = NotebookWrite, GDM = GoogleDistanceMatrix, AccomS = AccommodationSearch, Plan = Planner). Requests that differ in entities, constraints, or wording therefore often share the same underlying planning skeleton. This observation motivates storing complete historical plans as **workflow hyperedges**, which can be retrieved and instantiated for familiar requests.

3.2 Insight 2: Fragment-Level Redundancy

Insight 2. Even when complete plans differ, they frequently share recurring action fragments, motivating fragment-level composition and factorized storage.

Complete-workflow reuse alone cannot exploit partial overlap. Two requests may require different overall plans while sharing transportation, search, filtering, recording, or summarization routines. A memory that stores only complete plans would repeatedly store these common fragments and would still invoke the LLM when no full workflow matches.

Using the same 180 trajectories, we extract all contiguous action subsequences of length 2–6. Each fragment is counted at most once per plan and must contain at least two distinct action types, which removes trivial within-plan repetitions. We then measure the percentage of plans containing each fragment and report the five most frequent patterns.

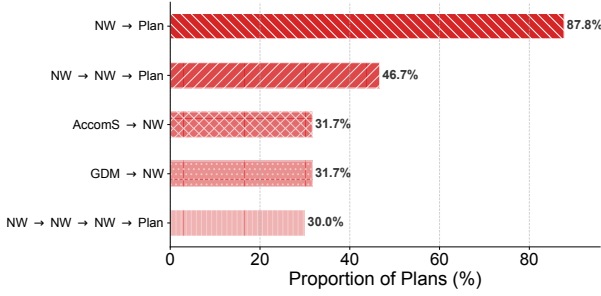


Figure 3: Top-5 recurring sub-plan fragments. Each bar shows the percentage of plans containing the given action subsequence, counted once per plan and requiring at least two distinct action types.

Figure 3 reveals substantially stronger redundancy at the fragment level. The fragment $NW \rightarrow Plan$ appears in 87.8% of plans, while $NW \rightarrow NW \rightarrow Plan$ appears in 46.7%. Tool-specific fragments such as $AccomS \rightarrow NW$ and $GDM \rightarrow NW$ each appear in 31.7%. Thus, workflows that cannot be reused as complete plans may still be assembled from recurring local structures.

These observations motivate **Persistent TinyEdges**, which factorize recurring fragments shared across workflow hyperedges. Persistent TinyEdges support partial-plan composition and store common structure only once. Their canonical identities also expose opportunities for **Transient TinyEdge Groups**, which share prompt prefixes among compatible active requests. The analysis here establishes the potential for fragment reuse; the runtime miner further applies support, closedness, and executability criteria before materializing a fragment.

Together, these observations motivate multi-granular planning reuse: complete workflows support direct reuse, while Persistent TinyEdges capture reusable structure below the workflow level.

4 Method

4.1 Problem Setup

We consider an agent that processes a stream of requests. For each request q , the agent constructs a plan $\pi = (V_\pi, \prec_\pi)$, where V_π contains typed executable operations and $\prec_\pi$ specifies their order and dependencies. Concrete arguments are supplied by the current request during parameter instantiation.

The agent maintains a planning memory that is updated from completed trajectories. Our goal is to reduce model calls, tokens, monetary cost, and latency. Task effectiveness is evaluated separately using the official metric of each benchmark.

4.2 Framework Overview

G-Agent separates plan acquisition from parameter instantiation and execution. As shown in Figure 4, it first attempts workflow reuse and then TinyEdge composition. When memory coverage is insufficient, it invokes the base LLM planner to generate a complete plan. A lightweight model generates the coarse and detailed plans used during composition. The selected executable plan is then instantiated with request-specific arguments. Compatible requests

may also form a Transient TinyEdge Group and share a common prompt prefix.

4.3 Multi-Granular Hypergraph Memory

G-Agent stores planning knowledge at two granularities over shared canonical nodes. It maintains a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E}^W, \mathcal{E}^T)$, where $\mathcal{V}$ contains canonical executable operations, $\mathcal{E}^W$ contains workflow hyperedges, and $\mathcal{E}^T$ contains Persistent TinyEdges. Each node $v \in \mathcal{V}$ is represented as $(T, D, \mathcal{M})$, where T is an operation or tool identifier, D is a normalized description, and $\mathcal{M}$ is its typed argument schema.

Workflow hyperedges and **Persistent TinyEdges** support different levels of reuse. A workflow hyperedge $e = (S_e, \prec_e, z_e)$ stores a complete ordered plan, its execution dependencies, and its retrieval embedding. A Persistent TinyEdge is represented as $t = (S_t, z_t, \sigma_t, \mathcal{P}_t)$, where S_t is an ordered subtask fragment, z_t is its retrieval embedding, σ_t is its support, and $\mathcal{P}_t$ records its source workflows. Workflow hyperedges support direct reuse, while Persistent TinyEdges support partial composition and compact storage.

A Persistent TinyEdge may be shared by multiple workflow hyperedges. This representation explicitly records cross-workflow structural overlap and stores each shared fragment only once. The source-workflow references also retain the provenance of each reusable fragment.

The memory separates reusable topology from request-specific values. Workflow hyperedges and Persistent TinyEdges are indexed by their embeddings, while TinyEdge support and provenance are stored as retrieval metadata. Locations, dates, budgets, preferences, and other runtime values are supplied by the current request during parameter instantiation.

4.4 Persistent TinyEdge Mining

G-Agent mines **Persistent TinyEdges** incrementally as new workflows are admitted to memory. When a newly completed trajectory is admitted as a workflow hyperedge, its executable steps are mapped to their canonical node representations. TinyEdges are mined only within contiguous executable segments; reasoning and control steps act as fragment boundaries.

Within each executable segment, G-Agent extracts contiguous fragments of length $[l_{\min}, l_{\max}]$ and counts how many admitted workflows contain each fragment. The support of fragment f is

$$\text{sup}(f) = |\{\pi \in \mathcal{P}_{\text{adm}} \mid f \text{ occurs contiguously in } \pi\}|,$$

where $\mathcal{P}_{\text{adm}}$ is the set of workflows admitted to memory. Each workflow contributes at most once to the support of a fragment, even if that fragment occurs multiple times in the same workflow.

Only frequent, closed, and executable fragments are materialized as Persistent TinyEdges. A fragment is frequent when $\text{sup}(f) \geq \gamma$. It is closed when no strict contiguous super-fragment has the same supporting workflow set. Executability requires all referenced operations to remain available.

When a new workflow is admitted, G-Agent updates only the affected fragment candidates and materializes those that satisfy the admission criteria. For a workflow of length L , fragment enumeration examines at most $O(Ll_{\max})$ contiguous windows and is therefore linear in L for a fixed $l_{\max}$.

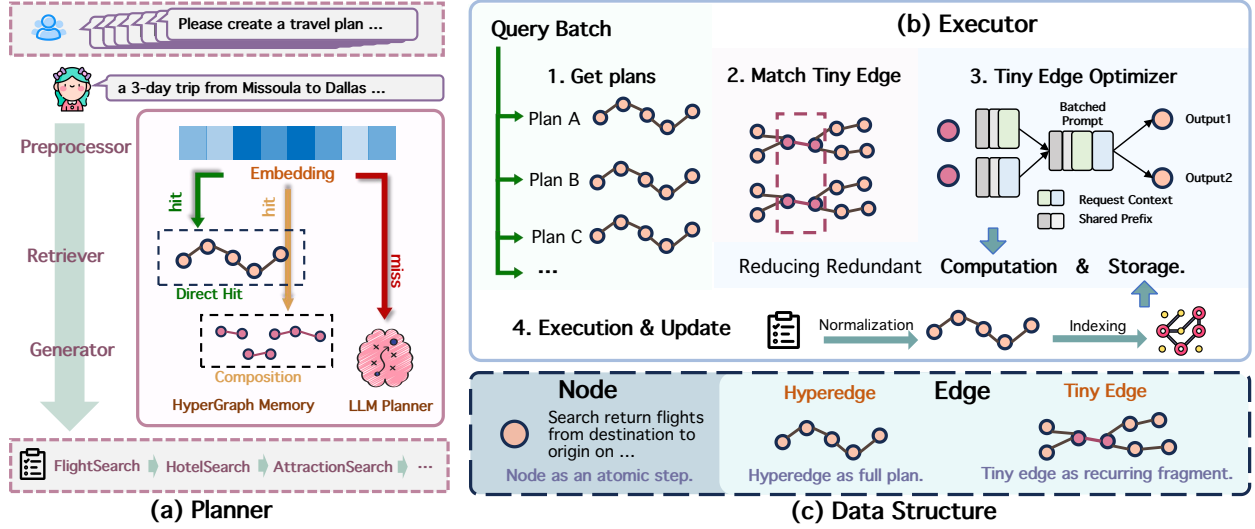


Figure 4: G-Agent framework. G-Agent obtains plans through workflow reuse or TinyEdge composition, and invokes the base LLM planner when memory coverage is insufficient. Persistent TinyEdges support composition and compact storage, while compatible requests form Transient TinyEdge Groups for shared-prompt batching.

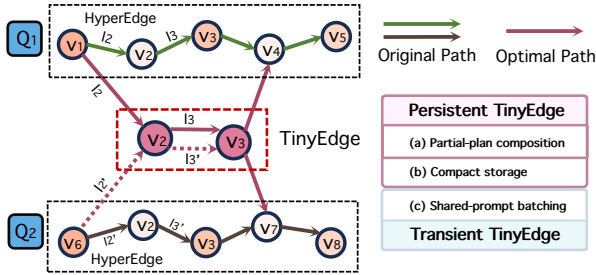


Figure 5: Two TinyEdge forms and their roles. Persistent TinyEdges support partial-plan composition and compact storage, while Transient TinyEdge Groups enable shared-prompt batching.

4.5 Plan Reuse and Composition

G-Agent first attempts complete workflow reuse. Given a request q , it retrieves the workflow hyperedge e^* with the highest matching score. If the score exceeds θ_W , G-Agent instantiates the retrieved workflow with request-specific values and follows the **Fast Path**.

When whole-plan reuse is unavailable, a lightweight model generates a coarse plan $S(q) = (s_1, \dots, s_m)$. Each s_i describes a high-level subgoal rather than a directly executable operation. The coarse plan specifies the task structure and guides the retrieval and ordering of reusable fragments.

Candidate retrieval. For each coarse subgoal s_i , G-Agent retrieves the top- k Persistent TinyEdges according to semantic similarity. Candidates below the similarity threshold θ_F are discarded. The remaining TinyEdges are treated as potentially relevant fragments for the corresponding subgoal.

TinyEdge selection and expansion. Let $\mathcal{T}_i$ denote the retained candidates for subgoal s_i . G-Agent selects the highest-scoring candidate

t_i^* from each non-empty $\mathcal{T}_i$. Composition proceeds only when every coarse subgoal has a selected fragment and at least two distinct Persistent TinyEdges are used. Conditioned on the selected fragments, the lightweight model expands each coarse subgoal into a detailed executable subplan. These subplans are assembled in the order of the coarse plan, and request-specific values are supplied during parameter instantiation.

LLM fallback. If neither whole-plan reuse nor TinyEdge composition provides sufficient coverage, G-Agent invokes the base LLM planner to generate a complete plan. The generated plan is then passed to the same downstream executor.

4.6 Transient TinyEdge Grouping

Transient TinyEdge Groups expose runtime sharing without modifying persistent memory. Active requests enter the same candidate group when they share a canonical local structure, operation type, compatible argument schema, and prompt-prefix template.

For a candidate group $\mathcal{G}$, G-Agent estimates its prompt-token saving as

$$\Delta_{\text{tok}}(\mathcal{G}) = \sum_{q_i \in \mathcal{G}} C_{\text{single}}(q_i) - C_{\text{group}}(\mathcal{G}),$$

where $C_{\text{single}}(q_i)$ is the prompt-token cost of processing request q_i independently, and $C_{\text{group}}(\mathcal{G})$ is the total prompt-token cost of grouped processing. Grouped processing encodes the common prefix once and retains the request-specific context required by each request, together with the batching-format overhead.

A candidate group closes when its size reaches B_{max} or the waiting window Δt expires. G-Agent executes the requests as a group only when $\Delta_{\text{tok}}(\mathcal{G}) > 0$. Otherwise, the requests proceed independently.

Algorithm 1: Workflow Reuse and Composition

Input: Request q , workflow memory $\mathcal{E}^W$, Persistent TinyEdge memory $\mathcal{E}^T$

Output: Plan π and route r

```

1  $(e^*, \rho) \leftarrow \text{RetrieveWorkflow}(q, \mathcal{E}^W);$ 
2 if  $\rho \geq \theta_W$  then
3    $\pi \leftarrow \text{Instantiate}(e^*, q);$ 
4   return  $(\pi, \text{FAST});$ 
5  $S = (s_1, \dots, s_m) \leftarrow \text{CoarsePlan}(q);$ 
6  $P \leftarrow [];$ 
7 for  $i \leftarrow 1$  to  $m$  do
8    $\mathcal{T}_i \leftarrow \text{RetrieveTinyEdges}(s_i, \mathcal{E}^T, k);$ 
9    $\mathcal{T}_i \leftarrow \{t \in \mathcal{T}_i \mid \text{sim}(s_i, t) \geq \theta_F\};$ 
10  if  $\mathcal{T}_i \neq \emptyset$  then
11     $t_i^* \leftarrow \arg \max_{t \in \mathcal{T}_i} \text{sim}(s_i, t);$ 
12     $P \leftarrow \text{Append}(P, (s_i, t_i^*));$ 
13 if  $|P| < m$  or  $\text{DistinctTinyEdges}(P) < 2$  then
14   return  $(\text{LLMPlan}(q), \text{FALLBACK});$ 
15  $\hat{S} \leftarrow \text{LightweightDetailPlan}(S, P, q);$ 
16  $\pi \leftarrow \text{AssembleInOrder}(\hat{S});$ 
17 return  $(\pi, \text{COMPOSITION});$ 

```

4.7 Online Update and Maintenance

G-Agent updates the hypergraph online as new trajectories are completed. Admitted trajectories are stored as workflow hyperedges and incrementally update fragment support and provenance for Persistent TinyEdge mining.

To reduce the influence of stale workflows and prevent unbounded hypergraph growth, G-Agent applies an LRU policy to evict long-inactive entries.

5 Experiments

To evaluate the effectiveness and efficiency of G-Agent, we conduct comprehensive experiments to answer the following questions:

- **RQ1 (Pareto Efficiency):** Can G-Agent reduce monetary and token costs while maintaining competitive task effectiveness?
- **RQ2 (Comparison with Efficient Agents):** How does G-Agent compare with representative memory-based and dual-process agents in task effectiveness and monetary cost?
- **RQ3 (System Scalability):** How does G-Agent scale with increasing request concurrency and planning-memory size?
- **RQ4 (Composition Mechanism):** How does TinyEdge composition extend structural reuse beyond complete-workflow matching, and how does it reduce generative planning cost?

5.1 Experimental Setup

Datasets. We evaluate G-Agent on four benchmarks spanning diverse planning complexities, task structures, and tool ecosystems: 1) **TravelPlanner** [27] is a challenging real-world travel-planning benchmark involving diverse practical constraints and planning requirements. 2) **Bamboogle** [19] is a multi-hop question-answering benchmark requiring agents to decompose complex

questions into interdependent search steps, evaluated using normalized reference matching followed by LLM-based judging. 3) **WorkBench** [22] is a realistic workplace-agent benchmark involving tool use and database-state modification. We report task success based on whether execution produces the expected final state. 4) **AppWorld** [23] is a large-scale benchmark for tool-using agents, featuring a broad and heterogeneous API ecosystem. We report Task Goal Completion (TGC) using the official state-based evaluator.

Baselines. We compare G-Agent against four representative baselines: (1) **ReAct** [31]: The standard interleaved reasoning-and-acting baseline (Generative, No Memory); (2) **Plan-and-Solve (P+S)** [26]: A two-stage prompting strategy (Generative, No Memory); (3) **SwiftSage** [14]: A dual-process agent using a small model for fast actions and a large model for deliberate reasoning; we implement its fast policy through in-context learning (Context Memory); and (4) **PlanCache** [32]: A retrieval-based agent that caches whole-plan templates (Retrieval-based, No Mining).

We use Qwen3-Max for large-model planning and reasoning, and Qwen2.5-7B-Instruct for intent extraction, parameter instantiation, and other lightweight tasks. On AppWorld, Qwen3.7-Plus serves as the execution model to handle its more demanding code-generation and API-use tasks. G-Agent indexes hyperedges in Chroma using normalized 1,024-dimensional embeddings from BAAI/bge-large-en-v1.5. We set both the maximum TinyEdge batch size and task concurrency to 8. Additional implementation details are provided in Appendices A and C.

We simulate a realistic online-serving scenario in which the agent **starts with an empty memory** and processes requests sequentially from the evaluation stream. This protocol fully accounts for cold-start planning overhead. During execution, the hypergraph memory is updated online: newly completed trajectories that satisfy the admission criteria are added to the workflow memory and become available for subsequent complete-workflow reuse and TinyEdge mining.

5.2 Main Results: Effectiveness vs. Cost

Table 2 shows that G-Agent improves the cost-effectiveness trade-off rather than uniformly maximizing task accuracy. It achieves the lowest monetary cost on all four benchmarks while maintaining competitive task effectiveness. G-Agent also outperforms PlanCache in both accuracy and cost across every benchmark, indicating that selective multi-granular reuse is more effective than whole-template caching.

The advantage is clearest on TravelPlanner. G-Agent achieves 80.69% accuracy, outperforming ReAct by 2.77 percentage points and approaching Plan-and-Solve at 82.36%. Meanwhile, it reduces cost by 84.7% relative to ReAct and by 62.7% relative to PlanCache. Travel requests differ in entities and constraints but repeatedly involve similar transportation, accommodation, attraction, and itinerary-construction routines, allowing G-Agent to reuse planning structures while instantiating request-specific values.

A similar pattern appears on WorkBench and AppWorld. On WorkBench, G-Agent outperforms Plan-and-Solve, SwiftSage, and PlanCache while achieving the lowest cost. On AppWorld, it ranks second to ReAct and exceeds the other planning and memory-based methods. These results suggest that recurring state-modification

Table 2: Main results on TravelPlanner, Bamboogle, WorkBench, and AppWorld. G-Agent reduces cost while achieving better accuracy than other memory-based approaches (SwiftSage and PlanCache).

Method	TravelPlanner		Bamboogle		WorkBench		AppWorld	
	Accuracy (↑)	Cost (\$ ↓)	Accuracy (↑)	Cost (\$ ↓)	Accuracy (↑)	Cost (\$ ↓)	Accuracy (↑)	Cost (\$ ↓)
ReAct	77.92%	21.3788	67.20%	0.6151	79.09%	0.2465	51.19%	21.9041
Plan-and-Solve	82.36%	4.4623	63.20%	0.4545	66.34%	0.3271	41.67%	4.3275
SwiftSage	68.47%	9.857	67.20%	0.3792	65.34%	0.3283	37.50%	6.018
PlanCache	72.83%	8.768	58.40%	0.9898	30.00%	0.2255	30.36%	4.499
G-Agent (Ours)	80.69%	3.2675	66.40%	0.3671	75.45%	0.2193	44.64%	4.0924

Table 3: Accepted reuse quality across benchmarks.

Dataset	Method	Accepted Reuse	Accuracy	
			Reuse	Non-Reuse
TravelPlanner	PlanCache	57.11%	71.20%	75.00%
	G-Agent	82.78%	80.87%	79.84%
Bamboogle	PlanCache	90.40%	24.78%	75.00%
	G-Agent	59.20%	72.97%	58.00%
WorkBench	PlanCache	90.00%	25.25%	72.73%
	G-Agent	82.73%	78.02%	63.16%
AppWorld	PlanCache	38.69%	18.46%	37.86%
	G-Agent	20.24%	79.41%	35.82%

and API-use structures can be reused effectively, although fresh reasoning remains valuable for novel state-dependent operations.

Bamboogle presents a different pattern. G-Agent achieves 66.4% accuracy, only 0.8 percentage points below ReAct and SwiftSage, while outperforming Plan-and-Solve and PlanCache at the lowest cost. Since Bamboogle depends heavily on query-specific evidence retrieval and multi-hop fact integration, planning reuse reduces cost but provides a smaller effectiveness advantage.

Table 3 further demonstrates the reliability of G-Agent’s reuse decisions. On TravelPlanner, it accepts 82.78% of requests for reuse, with comparable accuracy on reused and non-reused cases (80.87% vs. 79.84%). On Bamboogle, WorkBench, and AppWorld, G-Agent accepts fewer reuse cases than PlanCache but achieves substantially higher reuse accuracy: 72.97% vs. 24.78%, 78.02% vs. 25.25%, and 79.41% vs. 18.46%, respectively. This shows that selective routing and multi-granular reuse avoid unreliable whole-template transfer.

On WorkBench, G-Agent combines high reuse coverage (82.73%) with higher accuracy on reused than non-reused requests (78.02% vs. 63.16%). On AppWorld, it accepts only 20.24% of requests for reuse, but these cases achieve 79.41% accuracy. Overall, G-Agent balances reuse coverage with reuse reliability rather than simply maximizing the hit rate.

Taken together, Tables 2 and 3 show that G-Agent achieves a stronger cost–effectiveness trade-off than whole-template caching by balancing reuse coverage, reuse reliability, and generative fallback.

5.3 System Scalability Analysis

We evaluate G-Agent’s scalability from two complementary perspectives: whether increasing concurrency creates additional shared-prefix opportunities, and whether TinyEdge factorization controls storage growth as the planning memory expands.

Request-side scalability. Transient TinyEdge Groups batch compatible requests under a shared prompt, allowing common instructions and tool schemas to be encoded once. We vary a joint scale parameter $B \in \{1, 2, 4, 8, 16\}$, which controls request concurrency, active-plan capacity, maximum group size, and API concurrency. As shown in Figure 6, increasing B from 2 to 8 raises the grouping rate from 8.10% to 32.83%. Relative to the $B = 1$ setting, prompt usage at $B = 8$ decreases from 3,291 to 2,991.84 tokens per item, a 9.1% reduction, while Commonsense Micro remains between 80.00% and 82.50%. The gain therefore comes from eliminating repeated prompt prefixes rather than reducing the amount of task-specific information processed.

At $B = 16$, the grouping rate falls slightly to 29.57%, and prompt usage rises to 3,005.93 tokens per item. The workload contains only a limited number of requests with compatible local structures. Once the high-frequency structures have been grouped, additional concurrent requests are increasingly heterogeneous and provide fewer new sharing opportunities. The request-side benefit therefore begins to saturate around $B = 8$.

Memory-side scalability. To evaluate memory-side scalability beyond the scale supported by the collected traces, we construct semi-synthetic graphs calibrated using the plan length, node novelty, fragment support, and reuse patterns observed in successful TravelPlanner trajectories. Figure 7(a) shows that the storage reduction grows from 14.4% at 10K plan-step references to 50.8% at 500K. This increasing benefit arises because each recurring fragment is stored once and referenced by an increasing number of workflows, allowing its storage cost to be amortized as memory grows.

At 500K references, TinyEdge-Compact reduces the SQLite footprint from 95.105 MB to 46.819 MB, while increasing p95 retrieval-and-reconstruction latency by only 0.100 ms. It preserves lossless reconstruction and returns the same top- k workflows as FullEdge-Only. Thus, TinyEdge factorization changes the storage-growth rate without altering retrieval semantics. As this experiment uses calibrated semi-synthetic graphs, it evaluates representation-level scalability rather than end-to-end serving performance.

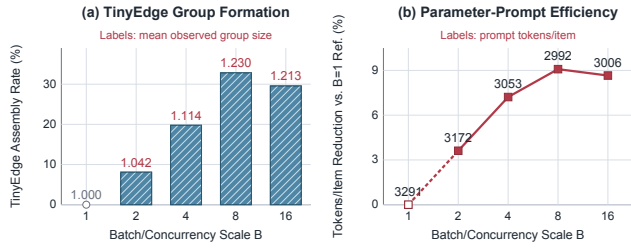


Figure 6: Request-side scalability. TinyEdge grouping rate and prompt tokens per item across batch/concurrency scales.

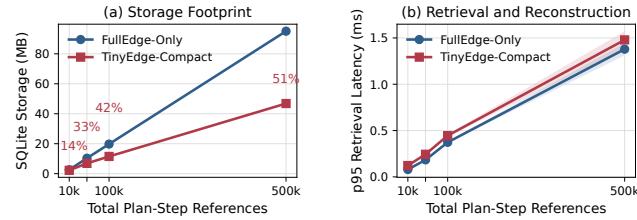


Figure 7: Memory-side scalability. Storage footprint and p95 retrieval-and-reconstruction latency across planning-memory scales.

5.4 Ablation Study

Table 4 isolates the contributions of two core reuse mechanisms: long-hyperedge matching for complete-workflow reuse and TinyEdge composition for partial-plan reuse. All variants use the same models, prompts, planner, executor, scheduler, concurrency, and execution budget.

Compared with the no-memory control, Full G-Agent reduces cost from \$1.911 to \$1.194, a 37.5% saving, while accuracy decreases from 84.23% to 81.67%. Planning memory therefore trades a limited effectiveness reduction for substantially cheaper structural reuse.

Removing Composition lowers the hit rate from 83.33% to 42.22% and increases cost from \$1.194 to \$1.463, showing that Composition provides most of the additional reuse coverage and reduces cost by 18.4%. However, accuracy rises from 81.67% to 83.47%, so Composition improves coverage and cost rather than accuracy. Remaining errors likely arise from fragment ordering, parameter instantiation, or cross-fragment consistency.

Disabling workflow-hyperedge reuse reduces the hit rate only slightly, from 83.33% to 82.22%, because Composition still covers most reusable requests. However, the cost increases from \$1.194 to \$1.341. This indicates that long hyperedges mainly provide a direct and low-overhead route for close matches, while TinyEdge composition supplies broader coverage for partial matches. The two mechanisms are therefore complementary.

5.5 Robustness Analysis

Table 5 compares G-Agent with SwiftSage and PlanCache under three backbone models. G-Agent achieves the lowest monetary cost in all settings, reducing cost relative to PlanCache by 72.9% with

Table 4: Ablation results on TravelPlanner.

Variant	Accuracy (↑)	Hit Rate (↑)	Cost (\$ ↓)
w/o memory	84.23%	0.00%	1.911
w/o Long-Hyperedge	81.94%	82.22%	1.341
w/o Composition	83.47%	42.22%	1.463
Full G-Agent	81.67%	83.33%	1.194

Table 5: Robustness across LLM backbones on TravelPlanner.

Backbone	Method	Acc. (↑)	Cost (\$ ↓)
Qwen3-Max	SwiftSage	68.47%	9.857
	PlanCache	72.83%	8.768
	G-Agent	77.78%	2.374
Llama-4-Maverick	SwiftSage	64.69%	0.7372
	PlanCache	65.78%	0.5108
	G-Agent	72.36%	0.3281
GLM-5.2	SwiftSage	70.31%	1.487
	PlanCache	60.16%	0.7073
	G-Agent	81.81%	0.6947

Qwen3-Max, 35.8% with Llama-4-Maverick, and 1.8% with GLM-5.2. Although the cost margin varies across model families and pricing regimes, retrieval-first routing consistently avoids repeated generative planning.

G-Agent also achieves the highest accuracy under all three backbones. It outperforms the strongest baseline by 4.95, 6.58, and 11.50 percentage points with Qwen3-Max, Llama-4-Maverick, and GLM-5.2, respectively. These results show that G-Agent provides consistent cost-effectiveness gains across different backbone models.

6 Conclusion

We presented G-Agent, a retrieval-first agent that externalizes historical planning traces as an annotated hypergraph memory. Workflow hyperedges enable complete-plan reuse, Persistent TinyEdges support fragment-level composition and compact storage, and Transient TinyEdge Groups reduce repeated prompt processing across compatible requests. G-Agent further integrates selective routing, generative fallback, and online memory updates to balance reuse efficiency with task reliability. Across four benchmarks, G-Agent consistently reduces monetary cost while preserving competitive task effectiveness. Ablation results show that workflow-hyperedge reuse provides a direct low-overhead path for familiar requests, whereas TinyEdge composition substantially expands reuse coverage and reduces generative planning cost. Scalability experiments further demonstrate that TinyEdge factorization controls memory growth with limited retrieval overhead. Overall, structured planning memory can amortize expensive planning when workloads contain recurring executable structure, while generative fallback remains necessary for novel requests or tasks dominated by instance-specific reasoning.

References

- [1] Rakesh Agrawal and Ramakrishnan Srikant. 1994. Fast Algorithms for Mining Association Rules in Large Databases. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*. 487–499.
- [2] Ammar Ahmed, Azal Ahmad Khan, Ayaan Ahmad, Sheng Di, Zirui Liu, and Ali Anwar. 2026. Retrieval-of-Thought: Efficient Reasoning via Reusing Thoughts. In *The Fourteenth International Conference on Learning Representations*.
- [3] Song Bai, Feihu Zhang, and Philip H. S. Torr. 2021. Hypergraph Convolution and Hypergraph Attention. *Pattern Recognition* 110 (2021), 107637. doi:10.1016/j.patcog.2020.107637
- [4] Claude Berge. 1989. *Hypergraphs: Combinatorics of Finite Sets*. North-Holland.
- [5] Edward Y Chang and Longling Geng. 2025. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4874–4886.
- [6] Zhangtao Cheng, Yuhao Ma, Jian Lang, Kumpeng Zhang, Ting Zhong, Yong Wang, and Fan Zhou. 2025. Generative Thinking, Corrective Action: User-Friendly Composed Image Retrieval via Automatic Multi-Agent Collaboration. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 334–344.
- [7] Xinshun Feng, Xinhao Song, Lijun Li, Gongshen Liu, and Jing Shao. 2026. SEARL: Joint Optimization of Policy and Tool Graph Memory for Self-Evolving Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 24518–24535.
- [8] Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. 2019. Hypergraph Neural Networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 3558–3565. doi:10.1609/aaai.v33i01.33013558
- [9] Jiawei Han, Jian Pei, and Yiwen Yin. 2000. Mining Frequent Patterns without Candidate Generation. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1–12. doi:10.1145/335191.335372
- [10] Jingyi Jia and Qibin Li. 2026. AutoTool: Efficient tool selection for large language model agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 31265–31273.
- [11] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. doi:10.1109/TBDATA.2019.2921572
- [12] Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Hofit Bata, Yoav Levine, Kevin Leyton-Brown, et al. 2022. MRKL Systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. *arXiv preprint arXiv:2205.00445* (2022).
- [13] Lingjing Kong, Xin Liu, Guangyi Chen, Martin Q. Ma, Xiangchen Song, Yuekai Sun, Mikhail Yurochkin, Taylor W. Killian, Ruslan Salakhutdinov, Kun Zhang, Eric P. Xing, and Zhengzhong Liu. 2026. From Reasoning Traces to Reusable Modules: Understanding Compositional Generalization in Language Model Reasoning. In *Proceedings of the 43rd International Conference on Machine Learning*.
- [14] Bill Yuchen Lin, Yicheng Fu, Karina Yang, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Prithviraj Ammanabrolu, Yejin Choi, and Xiang Ren. 2023. Swift-sage: A generative agent with fast and slow thinking for complex interactive tasks. *Advances in Neural Information Processing Systems* 36 (2023), 23813–23825.
- [15] Heikki Mannila, Hannu Toivonen, and A. Inkeri Verkamo. 1997. Discovery of Frequent Episodes in Event Sequences. *Data Mining and Knowledge Discovery* 1, 3 (1997), 259–289. doi:10.1023/A:1009748302351
- [16] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Ali Mousavi, Ihab F. Ilyas, Umar Farooq Minhas, Jeffrey Pound, and Theodoros Rekatsinas. 2023. High-Throughput Vector Similarity Search in Knowledge Graphs. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–25. doi:10.1145/3589777
- [17] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.
- [18] Jian Pei, Jiawei Han, Behzad Mortazavi-Asl, Jianyong Wang, Helen Pinto, Qiming Chen, Umeshwar Dayal, and Mei-Chun Hsu. 2001. PrefixSpan: Mining Sequential Patterns Efficiently by Prefix-Projected Pattern Growth. In *Proceedings 17th International Conference on Data Engineering (ICDE)*. 215–224. doi:10.1109/ICDE.2001.914830
- [19] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. 2023. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 5687–5711.
- [20] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems* 36 (2023), 68539–68551.
- [21] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems* 36 (2023), 8634–8652.
- [22] Olly Styles, Sam Miller, Patricio Cerda-Mardini, Tanaya Guha, Victor Sanchez, and Bertie Vidgen. [n. d.]. WorkBench: a Benchmark Dataset for Agents in a Realistic Workplace Setting. In *First Conference on Language Modeling*.
- [23] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 16022–16076.
- [24] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. [n. d.]. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research* ([n. d.]).
- [25] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science* 18, 6 (2024), 186345. doi:10.1007/s11704-024-40231-1
- [26] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*. 2609–2634.
- [27] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. 2024. TravelPlanner: A Benchmark for Real-World Planning with Language Agents. In *International Conference on Machine Learning*. PMLR, 54590–54613.
- [28] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. 2023. ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models. arXiv:2305.18323. <https://arxiv.org/abs/2305.18323>
- [29] Xifeng Yan and Jiawei Han. 2002. gSpan: Graph-Based Substructure Pattern Mining. In *Proceedings of the 2002 IEEE International Conference on Data Mining (ICDM)*. 721–724. doi:10.1109/ICDM.2002.1184038
- [30] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
- [31] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
- [32] Qizheng Zhang, Michael Wornow, and Kunle Olukotun. 2025. Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 38. 103270–103296.
- [33] Yunan Zhang, Shige Liu, and Jianguo Wang. 2024. Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL. In *Proceedings of the 40th IEEE International Conference on Data Engineering (ICDE)*.
- [34] Dengyong Zhou, Jiayuan Huang, and Bernhard Schölkopf. 2006. Learning with Hypergraphs: Clustering, Classification, and Embedding. In *Advances in Neural Information Processing Systems (NeurIPS)*.

A Experimental Details

A.1 Experimental Environment

We conduct all experiments on an Ubuntu 22.04.1 server equipped with a 6-core CPU, 25 GB of RAM, and a 32 GB NVIDIA Tesla V100 GPU.

A.2 API Pricing

Table 6 reports the API prices used for cost estimation in our experiments, in USD per million tokens.

Table 6: API pricing used for cost estimation (USD per 1M).

Model	Input	Output
Qwen2.5-7B-Instruct	\$0.04	\$0.10
Qwen3.7-Plus	\$0.40	\$1.60
Qwen3-Max	\$1.20	\$6.00
Llama-4-Maverick	\$0.20	\$0.80
GLM-5.2	\$0.76	\$2.42

A.3 Benchmark Evaluation Protocols

TravelPlanner. We evaluate generated itineraries using the official TravelPlanner constraint evaluator and report only the *Commonsense Constraint Micro Pass Rate*. The evaluator checks eight practical requirements, including valid locations and entities, reasonable routes, non-conflicting transportation, diverse restaurants and attractions, valid accommodation, and completeness. The metric is the fraction of satisfied commonsense constraints across all tasks; missing or malformed plans fail all applicable checks.

WorkBench Calendar. We use WorkBench’s outcome-centric evaluation. Each predicted action sequence is executed from the benchmark’s initial sandbox state, and a task succeeds only if the resulting database state matches that produced by the ground-truth actions. Execution errors, missing predictions, and unintended state changes are counted as failures. We report the resulting task success rate.

Bamboogle. Bamboogle tasks require an agent to answer a multi-hop factual question by performing one or more interdependent web searches. We do not evaluate Bamboogle using literal exact match. Instead, we use a two-stage answer correctness protocol. First, both the reference and predicted answers are normalized by lowercasing and removing punctuation. A prediction receives a score of 1.0 if it contains the normalized reference answer. Predictions that do not pass this rule are evaluated by an LLM judge, which assigns one of three scores: 1.0 for a fully correct answer, 0.5 for a partially correct answer, and 0.0 for an incorrect answer. Internal control messages, such as an unresolved replanning request, receive 0.0. We report *strict accuracy*, i.e., the fraction of tasks receiving 1.0, as the primary metric, and additionally report the fraction receiving at least 0.5. Because the first stage is based on reference containment, we separately audit ambiguous substring matches.

AppWorld. AppWorld provides stateful simulated applications that agents operate by iteratively generating and executing Python code containing application API calls. Tasks may require reading

information, coordinating operations across multiple applications, executing complex control flow, and modifying persistent application state. The official evaluator applies programmatic assertions to the resulting database state, checking both that all required effects occurred and that the agent introduced no unintended changes. This outcome-based evaluation allows different valid programs and API-call sequences to solve the same task.

AppWorld defines two official metrics. *Task Goal Completion* (TGC) is the percentage of tasks for which all evaluation assertions pass. *Scenario Goal Completion* (SGC) is the percentage of three-task scenarios for which every constituent task succeeds, and therefore measures robustness across related task variants. We use TGC as the primary metric. SGC is reported only for experiments covering complete scenarios. We report runtime execution success separately, since successful code execution or a call to `complete_task` does not imply that the final application state passes the official evaluator.

A.4 Replan Model Across Experiments

The replan model is not uniform across the experiments reported in this paper, and we disclose the distinction here for transparency. In the accepted-reuse study (Table 3), the TravelPlanner ablation (Table 4), the TinyEdge scalability study (Figure 6), and the WorkBench, Bamboogle, and AppWorld evaluations, the replanning step—invoked when neither Fast Path nor Composition provides sufficient coverage—is served by the large backbone model (Qwen-Max). In contrast, the main-method TravelPlanner results in Table 2 and the sensitivity analysis in this section (Table 6) adopt a small-model replan configuration: the recovery step is routed to Qwen-2.5-7B-Instruct, falling back to the large backbone only when the small model fails to produce a valid recovery plan. The two configurations differ solely in the replan model; the backbone, intent-extraction model, final-reasoning model, scheduler, concurrency, and acceptance threshold (θ_w) are otherwise identical. Readers should therefore interpret the main-table TravelPlanner numbers (Table 2) and the sensitivity numbers (Table 6) as obtained under small-model replan, and the remaining tables and figures as obtained under large-model replan.

B Case Study

We present representative TravelPlanner traces to illustrate the three planning routes in G-Agent: complete workflow reuse, TinyEdge composition, and LLM fallback. These traces were collected from a composition-oriented stress test and are used to explain the observed routing outcomes. The final architecture applies these routes in the order of Fast reuse, Composition, and LLM fallback.

Figure 8 overviews the full G-Agent pipeline, with case-study annotations highlighted in dashed boxes alongside each route.

Successful composition. The request in `validation_0012` asks for a three-day trip from St. Petersburg to Appleton with a budget of \$1,200. G-Agent did not reuse a complete historical workflow. Instead, it selected two Persistent TinyEdges and followed the Composition path.

Together, the selected fragments provided reusable structure for outbound and return transportation, a driving alternative, accommodation, attractions, and restaurants. The lightweight model expanded the retrieved fragments into a detailed plan in the order

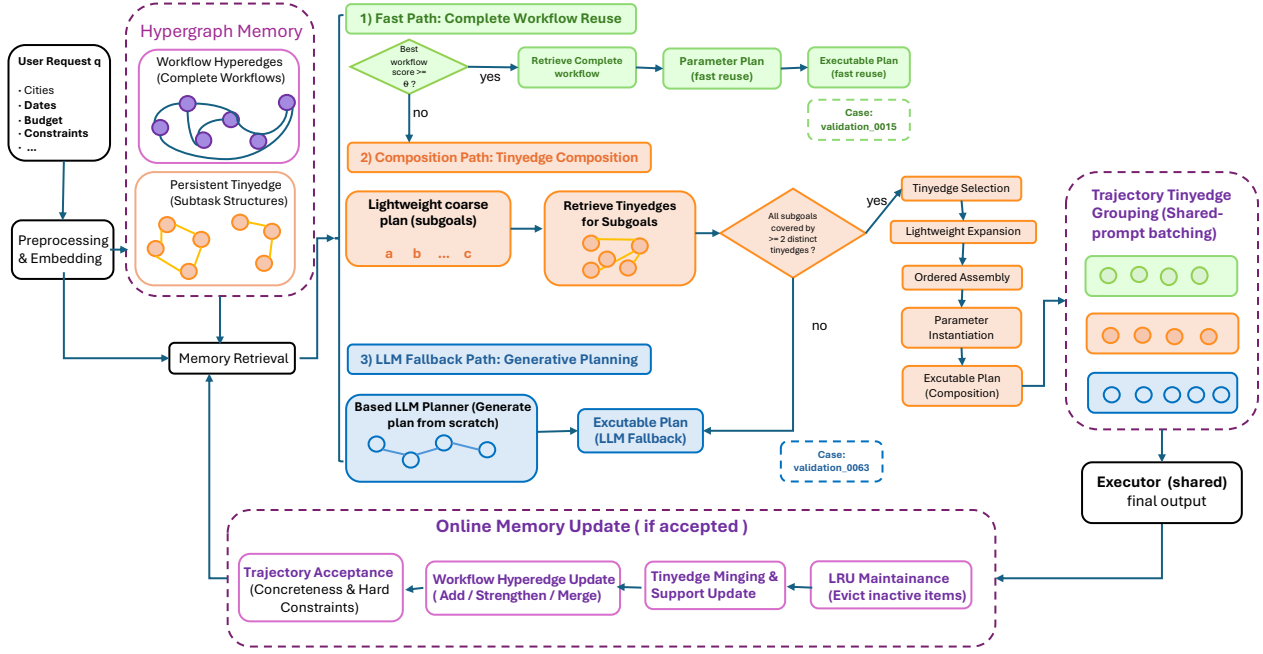


Figure 8: G-Agent Workflow. overviews the full G-Agent pipeline, with case-study annotations highlighted in dashed boxes alongside each route.

specified by the coarse plan. The current cities, dates, and budget were supplied during parameter instantiation rather than copied from the source workflows.

The resulting plan passed all eight commonsense checks and the applicable hard constraint. This case illustrates the role of Composition between the two simpler routes: no complete workflow was suitable for direct reuse, but multiple historical fragments provided sufficient structure to avoid invoking the base LLM planner.

Failure case. The request in validation_0033 asks for a five-day trip from Denver to two cities in Washington. G-Agent composed two Persistent TinyEdges covering city selection, transportation, accommodation, attractions, and restaurants. Although the resulting plan passed seven of eight commonsense checks, it failed the official evaluator with Invalid City Number. This case shows that structural coverage does not guarantee complete task correctness, especially when a request contains coupled multi-city constraints. Persistent TinyEdges provide reusable execution structure, but the detailed plan may still mishandle task-specific constraints.

Overall, the cases demonstrate the intended division of labor in G-Agent. Fast reuse handles requests that match a complete historical workflow, Composition combines fragments when only partial structure is reusable, and the base LLM planner handles requests that cannot be sufficiently covered by memory.

C Sensitivity Analysis

We study the sensitivity of the Fast Path acceptance threshold θ_W on TravelPlanner. As shown in Table 7, the threshold controls the trade-off among **task effectiveness**, **reuse coverage**, and **planning cost**.

A lower threshold accepts more retrieved workflows. At $\theta_W = 0.82$, G-Agent achieves the highest hit rate of **89.03%**, but its success rate is lower and its cost is the highest among the tested settings. This suggests that overly permissive routing may accept less reliable workflow matches.

Increasing the threshold generally reduces the hit rate and shifts more requests to Composition or LLM fallback. Although $\theta_W = 0.87$ achieves the lowest cost, its success rate drops to 72.01%. At $\theta_W = 0.85$, G-Agent achieves the highest success rate of 77.78%, while retaining a high hit rate of 88.89% at a moderate cost of \$2.374.

Overall, the results show that G-Agent is reasonably stable across the tested threshold range. We select $\theta_W = 0.85$ because it provides the best balance between **reuse efficiency** and **task reliability**.

Table 7: Sensitivity of the Fast Path acceptance threshold θ_W on TravelPlanner.

Threshold	Success Rate ($\uparrow$)	Cost (\$ $\downarrow$)	Hit Rate ($\uparrow$)
0.82	75.14%	3.345	89.03%
0.85	77.78%	2.374	88.89%
0.87	72.01%	2.231	81.67%
0.90	73.40%	2.536	77.22%
0.93	73.26%	3.030	77.78%

D Bounded Memory Maintenance

Motivation and scope. G-Agent continuously admits new workflow hyperedges and updates fragment statistics as requests are processed. Without an explicit memory budget, the number of stored workflows and fragment candidates may grow with the request stream. We therefore specify a workflow-level least-recently-used (LRU) policy that bounds the persistent planning memory under long-running deployment. The current benchmark streams do not create sustained memory pressure, so we use this policy to define the behavior of G-Agent under a fixed memory budget and do not attribute the effectiveness or efficiency gains in the main experiments to eviction.

Workflow-level eviction. Let $\mathcal{E}^W$ denote the workflow-hyperedge memory and let C_W be its maximum capacity. G-Agent maintains an LRU order over workflow hyperedges. A workflow is marked as recently used when it is newly admitted or successfully retrieved for reuse. When a Persistent TinyEdge is selected during composition, the workflows recorded in its provenance are also treated as referenced. If $|\mathcal{E}^W| > C_W$ after an admission, G-Agent repeatedly evicts the least recently used workflow until the capacity constraint is restored. Although C_W is expressed as a workflow count for clarity, the same procedure can use a byte-level storage budget.

Consistency with Persistent TinyEdges. Evicting a workflow affects the fragment statistics from which Persistent TinyEdges are materialized. G-Agent therefore maintains a reverse index from each workflow to the bounded-length fragment candidates extracted from that workflow. When a workflow is removed, its identifier is deleted from the provenance set of every affected fragment and the corresponding support count is decremented. G-Agent then locally reevaluates the admission conditions of the affected fragments and their contiguous super-fragments. A fragment remains materialized as a Persistent TinyEdge only if it is frequent, closed, and executable under the updated memory. Fragments that no longer satisfy these conditions are removed, while newly closed fragments may be materialized from the maintained candidate statistics. Canonical operation nodes that are no longer referenced by any workflow hyperedge or Persistent TinyEdge are garbage-collected.

Maintenance operations. $\text{UpdateStatistics}(\mathcal{A}, e, +1)$ adds workflow e to the provenance set of each fragment in $\mathcal{A}$ and increments its support, whereas $\text{UpdateStatistics}(\mathcal{A}, e, -1)$ performs the inverse operation. $\text{RefreshTinyEdges}(\mathcal{A}, \gamma)$ reevaluates frequency, closedness, and executability for the affected fragments and their

Algorithm 2: Workflow-Level LRU Maintenance

Input: Workflow memory $\mathcal{E}^W$, TinyEdge memory $\mathcal{E}^T$
Input: Capacity C_W , support threshold γ

```

1 Procedure  $\text{ADMITWORKFLOW}(e)$ ;
2   Insert  $e$  into  $\mathcal{E}^W$  and mark it as most recently used;
3    $\mathcal{F} \leftarrow \text{ExtractFragments}(e)$ ;
4    $\text{UpdateTinyEdges}(\mathcal{F}, e, +1, \gamma)$ ;
5   while  $|\mathcal{E}^W| > C_W$  do
6      $e_{\text{old}} \leftarrow \text{PopLRU}(\mathcal{E}^W)$ ;
7      $\mathcal{F}_{\text{old}} \leftarrow \text{ExtractFragments}(e_{\text{old}})$ ;
8      $\text{UpdateTinyEdges}(\mathcal{F}_{\text{old}}, e_{\text{old}}, -1, \gamma)$ ;

```

bounded-length contiguous super-fragments. The procedure therefore preserves the same TinyEdge admission criteria used during incremental mining rather than retaining fragment statistics from already evicted workflows.

Complexity and memory bound. Using a hash table and a doubly linked list, touching a workflow and identifying the least recently used workflow require $O(1)$ time. For an evicted workflow of length L , extracting and updating all contiguous fragments of length at most $l_{\max}$ requires at most $O(Ll_{\max})$ operations. Local closedness maintenance additionally examines only bounded-length super-fragments of the affected candidates. Therefore, for a fixed $l_{\max}$, the maintenance cost is linear in the workflow length. The number of stored workflow hyperedges is bounded by C_W , and the associated fragment-candidate state is bounded by $O(C_W \bar{L} l_{\max})$, where $\bar{L}$ is the average admitted workflow length.

Limitations. LRU provides a deterministic memory bound and gradually removes structures that have not been reused for a long period. However, recency is not a complete measure of semantic validity. In particular, an abrupt tool-interface or argument-schema change may invalidate a recently accessed workflow. G-Agent applies executability and schema validation before reuse, but production deployments may additionally require explicit interface versioning or invalidation signals. Evaluating alternative utility-aware eviction policies under long-running non-stationary workloads is left for future work.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009